ePS

EPARTICIPATION SYSTEM

Mobile App

Citizen Android/iOS client — Developer & Build Manual

Project: `eps-mobile-flutter` • Version 0.1.0+1

City Government of San Fernando, La Union

Compiled 9 September 2026

Contents

1. Introduction
- 1.1 Where the mobile app fits
- 1.2 Scope & feature parity

2. Technology Stack
- 2.1 Why Flutter
- 2.2 Packages

3. Project Structure

4. Configuration
- 4.1 Compile-time variables
- 4.2 The dart_define files
- 4.3 Choosing the right server URL

5. Prerequisites & First-time Setup
- 5.1 Install the Flutter SDK
- 5.2 Android toolchain
- 5.3 Generate the project scaffolding
- 5.4 Native tweaks to re-apply

6. Running the App
- 6.1 On an Android emulator
- 6.2 On a physical Android phone
- 6.3 On the web (quick UI check)

7. Building a Release
- 7.1 Signing key
- 7.2 Build commands
- 7.3 Versioning

8. Backend Contract
- 8.1 Authentication model
- 8.2 Endpoints used

9. Application Architecture
- 9.1 Layers
- 9.2 State & session
- 9.3 Routing & the auth redirect

10. Screens & Features

11. Real-time & Notifications
- 11.1 The socket
- 11.2 Notification triggers (backend)
- 11.3 The notification tray
- 11.4 Response ratings & office satisfaction
- 11.5 Cross-client parity

12. Design Decisions
- 12.1 Sign-in required to participate
- 12.2 Anonymous complaint filing kept
- 12.3 Splash screen
- 12.4 Launcher icon

13. Troubleshooting Log

14. Change Log (this engagement)

15. Quick Reference Card

1. Introduction

The **ePS Mobile App** (`eps-mobile-flutter`) is the citizen-facing Android and iOS client for the eParticipation System of the City Government of San Fernando. It was built with Flutter as the mobile counterpart of the existing citizen web client and talks to the same backend API.

1.1 Where the mobile app fits

The ePS platform is a multi-repository workspace under `D:\Development\eps-app` :

Folder	Role
`eps-api-express-js`	Express 5 + Sequelize (MySQL) REST API mounted under `/api` , plus a Socket.IO channel. JWT auth (4 h expiry) and an `api` key header.
`eps-www-quasar-external`	Citizen web client (Quasar v2 / Vue 3). File / track complaints, discussions, polls.
`eps-www-quasar-internal`	Staff / admin web client (the "Complaint Desk").
`eps-www-quasar-kiosk`	On-site kiosk client.
`eps-mobile-flutter`	**Citizen mobile app — this manual.** Mirrors the feature set of the external web client.

Because the mobile app and the external web client share one backend, any server-side change (for example a new notification trigger) reaches both automatically.

1.2 Scope & feature parity

The app implements the same citizen workflows as `eps-www-quasar-external` :

- **Authentication** — register, verify email by OTP, sign in.
- **Complaints (guest)** — file a complaint without an account (email OTP flow), verify the OTP, track a complaint by reference number + access code.
- **Complaints (account)** — file without OTP, "My Complaints" list, complaint detail with follow-ups and response ratings, claim a previously-filed complaint.
- **Discussions** — list, thread, comment (sign-in required).
- **Polls** — list, vote, results (sign-in required).
- **Real-time notifications** — socket-driven unread badge, notification tray with history.

2. Technology Stack

2.1 Why Flutter

Flutter was chosen (over Quasar/Capacitor, React Native, or native) after evaluating the trade-offs. The deciding factors, given a separate project folder was wanted:

- A single Dart codebase produces both Android and iOS binaries with a genuinely native feel.
- The API layer (Dio interceptors, Socket.IO client, models) maps cleanly onto the patterns already used in the Vue clients, so the contract is easy to keep in sync.
- No web-view bridge; direct access to camera, secure storage and push infrastructure.

2.2 Packages

Package	Version	Purpose / web-client equivalent
`provider`	^6.1.2	State via `ChangeNotifier` — maps to the Pinia stores.
`dio`	^5.7.0	HTTP client. Interceptors mirror `src/boot/axios.js`: `api` key header, bearer token, 401 → logout.
`go_router`	^14.6.2	Routing. `StatefulShellRoute` gives the authed area a bottom-nav shell.
`flutter_secure_storage`	^9.2.2	JWT stored in the Android keystore / iOS keychain.
`shared_preferences`	^2.3.3	User JSON + the persisted guest token.
`socket_io_client`	^3.0.2	Realtime channel. Mirrors `src/boot/socket.js`.
`image_picker`	^1.1.2	Evidence photos for complaints and follow-ups.
`cached_network_image`	^3.4.1	Thumbnail caching for served `/uploads` assets.
`http_parser`, `intl`	^4.0.2, ^0.19.0	Multipart MIME types; date formatting.
`flutter_native_splash` (dev)	^2.4.4	Generates the OS launch screen.
`flutter_lints` (dev)	^5.0.0	Lint rules (plus `prefer_single_quotes`, `require_trailing_commas`).

> **Dependency override:** `pubspec.yaml` pins `path_provider_android: 2.2.15`. Version 2.3.x switched to `jni`, which forces a ~1 GB Android NDK download at build time. Nothing in this app needs native code, so the older channel-based implementation is used instead. See §13.

3. Project Structure

Only `lib/`, `pubspec.yaml`, `test/`, `README.md` and the `dart_define*.json` templates are meant to be committed. The platform folders (`android/`, `ios/`, `web/`) are generated locally (see §5.3) and git-ignored.

```
lib/
  main.dart                    bootstrap: build ApiClient, load guest id,
                               create AuthController, wire MultiProvider
  src/
    app.dart                   MaterialApp.router + theme + GoRouter

    config/
      env.dart                 --dart-define reader (EPS_API_BASE_URL, EPS_API_KEY)
      app_config.dart          derived URLs (apiBaseUrl, socketUrl, resolveAsset)

    api/
      api_client.dart          Dio instance + interceptors
      api_exception.dart       normalized error type (flattens {message}/{error}/{errors[]})

    realtime/
      socket_service.dart      Socket.IO lifecycle, loadNotifications / force_logout

    models/
      user.dart  complaint.dart  discussion.dart  poll.dart
      app_notification.dart  json_utils.dart (lenient coercion helpers)

    services/                  one class per backend resource
      auth_service.dart  complaint_service.dart  discussion_service.dart
      poll_service.dart  notification_service.dart

    state/
      auth_controller.dart     ChangeNotifier: token/user/isAuthenticated,
                               socket lifecycle, notification count + list
      guest_identity.dart      per-install UUID for anonymous poll votes

    router/
      app_router.dart          GoRouter + auth redirect + bottom-nav shell

    theme/
      app_theme.dart           brand palette (#204888 primary, #FFC900 secondary),
                               statusColor() mapping

    ui/
      screens/                 one file per screen (17 screens)
      shell/home_shell.dart    bottom navigation container
      widgets/                 async_view, auth_scaffold, complaint_body,
                               evidence_gallery, notification_bell, otp_field,
                               photo_picker, sign_in_prompt, status_chip, format,
                               app_snack
```

4. Configuration

4.1 Compile-time variables

All backend configuration is injected at build/run time with `--dart-define`. There is **no in-app settings screen** for this — the values are compiled into the binary. They are read in `lib/src/config/env.dart`:

Key	Default	Notes
`EPS_API_BASE_URL`	`http://10.0.2.2:1000`	Backend origin, *without* the trailing `/api`. `10.0.2.2` is the Android emulator's alias for the host machine's `localhost`.
`EPS_API_KEY`	(the dev key)	Must equal `API_KEY` in the backend `.env`. Sent as the `api` header on every request (`AuthMiddleware.validateApiKey`).

4.2 The dart_define files

Rather than typing long `--dart-define` flags, use a JSON file with `--dart-define-from-file`. The repo carries these (the `.json` ones without `example` in the name are git-ignored):

File	Target	EPS_API_BASE_URL
`dart_define.example.json`	template (committed)	placeholder
`dart_define.json`	Android emulator	`http://10.0.2.2:1000`
`dart_define.device.json`	physical phone on the LAN	`http://<PC-LAN-IP>:1000` (e.g. `192.168.4.16`)
`dart_define.prod.json`	production release	`https://feedback.sanfernandocity.gov.ph`

4.3 Choosing the right server URL

> **The most common mistake:** building without `--dart-define-from-file`, so the binary falls back to the emulator default `10.0.2.2`. A physical phone cannot reach that address — every request times out with "*Cannot reach the server*". Always pass the file that matches your target.

Target	Define file	Build mode
Emulator	`dart_define.json` (`10.0.2.2`)	`--debug`
Physical phone on the LAN	`dart_define.device.json` (PC's Wi-Fi IPv4 — `ipconfig`, "Wireless LAN adapter Wi-Fi")	`--debug` or `--profile` (plain http)
Production / store	`dart_define.prod.json` (HTTPS API)	`--release`

> **Build mode is not optional here.** Only the *debug* and *profile* manifests permit cleartext `http://` (§5.4), so a phone talking to the LAN backend over `http` must be a `--debug` / `--profile` build. `--release` is for the HTTPS production API only. A release APK aimed at `http://192.168.x.x:1000` is blocked by Android and shows the same "Cannot reach the server" error.

5. Prerequisites & First-time Setup

5.1 Install the Flutter SDK

Flutter stable, Dart ≥ 3.6. On Windows:

```
# clone the stable channel
git clone https://github.com/flutter/flutter.git -b stable C:/src/flutter

# add to the User PATH (PowerShell)
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\src\flutter\bin", "User")

# reopen the terminal, then verify
flutter --version
flutter doctor
```

5.2 Android toolchain

1. Install **Android Studio** and complete its Setup Wizard (installs the SDK, platform-tools and emulator to `%LOCALAPPDATA%\Android\Sdk`).
2. **Settings → Languages & Frameworks → Android SDK → SDK Tools** — tick *Android SDK Command-line Tools (latest)*, *Platform-Tools*, *Android Emulator*.
3. **SDK Platforms** tab — install *Android API 36* (Flutter 3.47's compile SDK).
4. Accept licenses: `flutter doctor --android-licenses`.
5. Create a virtual device in **Device Manager**, or connect a physical phone (§6.2).

> On this machine the SDK ships the new `android` CLI ("cmdline-tools v23"), whose `sdkmanager.bat` shim crashes when Gradle calls it. If a build fails with `NTSTATUS 0xC0000409`, replace `cmdline-tools\latest` with the classic "Command line tools only" package. See §13.

5.3 Generate the project scaffolding

`flutter create .` fills in the native projects *without* touching `lib/` or `pubspec.yaml`:

```
cd D:\Development\eps-app\eps-mobile-flutter
flutter create . --project-name eps_mobile --org ph.gov.sanfernandocity --platforms=android,ios,web
flutter pub get
dart run flutter_native_splash:create      # regenerates the OS launch screen
dart run flutter_launcher_icons            # regenerates the launcher icon
flutter analyze                            # should report: No issues found!
```

5.4 Native tweaks to re-apply

These edits live inside the git-ignored `android/` and `ios/` folders, so **they must be redone after any `flutter create .`**:

File	Change	Why
`android/app/src/main/AndroidManifest.xml`	`android:label="ePS"` (default is the project name `eps_mobile`)	Home-screen app name.
`ios/Runner/Info.plist`	`CFBundleDisplayName` and `CFBundleName` → `ePS`	Home-screen app name (iOS).
`android/app/build.gradle.kts`	Comment out `ndkVersion = flutter.ndkVersion`	No plugin has native code; leaving it set makes AGP try to auto-provision the NDK during configuration and crash on the broken sdkmanager shim.
`android/app/build.gradle.kts`	Release signing block that reads `android/key.properties`, falling back to debug signing when absent	Lets `flutter build apk --release` work with or without a keystore.
`android/app/src/debug/AndroidManifest.xml` `.../profile/AndroidManifest.xml`	Add `<application android:usesCleartextTraffic="true" />`	Debug/profile builds can then reach a plain-`http` dev backend. Release builds still require HTTPS.
`android/gradle.properties`	`kotlin.incremental=false` `kotlin.incremental.useClasspathSnapshot=false`	The Pub cache (C:) and the project (D:) are on different drives; Kotlin's incremental compiler cannot build a relative path across drive roots and crashes with "different roots".

> **iOS** (`ios/Runner/Info.plist`): add `NSPhotoLibraryUsageDescription` and `NSCameraUsageDescription` for `image_picker` , and an ATS exception if pointing at a plain-http dev backend.

6. Running the App

In every case, start the backend first (`npm run dev` in `eps-api-express-js` , listens on port 1000).

6.1 On an Android emulator

```
cd D:\Development\eps-app\eps-mobile-flutter
flutter emulators --launch <emulator-id>          # or start it from Android Studio
flutter run --dart-define-from-file=dart_define.json
```

`dart_define.json` points at `10.0.2.2:1000` — the emulator's route to the host's `localhost` . Hot reload with `r` , quit with `q` .

6.2 On a physical Android phone

1. On the phone: **Settings** → **About phone** → tap *Build number* 7× → back → **Developer options** → enable *USB debugging*.
2. Connect with a **data** USB cable; accept the "Allow USB debugging?" prompt (tick "always allow").
3. Confirm it is seen: `flutter devices` lists it (e.g. `RMX2151` , id `KJAQBEHAFI9TYTYP`).
4. Phone and PC on the **same Wi-Fi**. Check the LAN IP in `dart_define.device.json` matches `ipconfig` ; update if it changed.
5. Run:

```
flutter clean          # if a previous build failed (clears stale Kotlin caches)
flutter run --debug -d KJAQBEHAFI9TYTYP --dart-define-from-file=dart_define.device.json
```

Or produce an installable APK:

```
flutter build apk --debug --target-platform android-arm64 ^
  --dart-define-from-file=dart_define.device.json
# output: build\app\outputs\flutter-apk\app-debug.apk
flutter install -d KJAQBEHAFI9TYTYP
```

> **Use `--debug` (or `--profile`) — not `--release` — when the backend is plain `http://` on the LAN.** The cleartext-HTTP exception is only in the *debug* and *profile* manifests (§5.4). A `--release` APK pointed at `http://192.168.x.x:1000` is blocked by Android before the request leaves — the symptom is a cleartext / network-security error, or the same "Cannot reach the server" message. A release build is only appropriate against the HTTPS production API (§7). To test a genuine release build over LAN `http` , you would first have to add `android:usesCleartextTraffic="true"` to the *main* manifest as well.

> **Wireless (Android 11+):** Developer options → *Wireless debugging* → *Pair device with pairing code*, then `adb pair <ip:port>` and `adb connect <ip:port>` , then unplug the cable.

6.3 On the web (quick UI check)

No Android SDK needed — useful for verifying layout and API calls:

```
flutter run -d chrome --dart-define=EPS_API_BASE_URL=http://localhost:1000
```

Note `localhost` here, not `10.0.2.2`. Secure storage on web falls back to a less-secure store and the build cannot target WebAssembly — both are fine for development, not for a web release.

7. Building a Release

7.1 Signing key

```
# one-time: create an upload keystore
keytool -genkey -v -keystore %USERPROFILE%\eps-upload-keystore.jks ^
  -keyalg RSA -keysize 2048 -validity 10000 -alias upload
```

Then create `android/key.properties` (git-ignored):

```
storeFile=C:/Users/Administrator/eps-upload-keystore.jks
storePassword=your-store-password
keyAlias=upload
keyPassword=your-key-password
```

`android/app/build.gradle.kts` picks this up automatically; without it, release builds fall back to debug signing (fine for internal testing, not for the Play Store).

7.2 Build commands

```
# Play Store upload (Android App Bundle)
flutter build appbundle --release --dart-define-from-file=dart_define.prod.json
#   -> build/app/outputs/bundle/release/app-release.aab

# Direct distribution (smaller, per-CPU APKs)
flutter build apk --release --split-per-abi --dart-define-from-file=dart_define.prod.json

# Single universal APK
flutter build apk --release --dart-define-from-file=dart_define.prod.json
```

> Release builds have **no cleartext-HTTP exception**, so `EPS_API_BASE_URL` in `dart_define.prod.json` must be `https://`.

7.3 Versioning

`version: 0.1.0+1` in `pubspec.yaml`. The part before `+` is the user-visible version name; the part after is the build number and must increase for every Play Store upload.

7.4 iOS

Requires a Mac with Xcode and an Apple Developer account: `flutter build ipa --release --dart-define-from-file=dart_define.prod.json`. Set the bundle id and signing team in `ios/Runner.xcworkspace`.

8. Backend Contract

8.1 Authentication model

- Every request carries the header `api: <EPS_API_KEY>` (`validateApiKey`).
- Authenticated requests add `Authorization: Bearer <jwt>` . The JWT lifetime is 4 hours.
- A 401/403 whose body mentions a token or API-key problem clears the local session and returns the user to the landing screen (Dio `onError` interceptor → `AuthController._onUnauthorized`).
- Guest complaint filing is a two-step OTP flow. Signed-in filing skips OTP.
- Guest poll votes are de-duplicated by a client-generated `guestToken` persisted per install (`lib/src/state/guest_identity.dart`).

8.2 Endpoints used

Method & path	Service method	Notes
`POST /citizen/register`	AuthService.register	emails an OTP
`POST /citizen/verify`	AuthService.verifyEmail	→ {user, token}
`POST /citizen/resend`	AuthService.resendVerification	
`POST /citizen/forgot-password`	AuthService.forgotPassword	{email}; enumeration-safe, emails a 6-digit code
`POST /citizen/verify-reset-code`	AuthService.verifyResetCode	{email, code}; checks without consuming
`POST /citizen/reset-password`	AuthService.resetPassword	{email, code, newPassword}; does NOT sign in
`POST /citizen/login`	AuthService.login	403 `notVerified` → verify screen
`POST /logout`	AuthService.logout	best-effort
`GET /citizen/complaints`	ComplaintService.myComplaints	auth
`POST /complaint/otp/send`	fileGuestComplaint	multipart
`POST /complaint/otp/validate`	validateComplaintOtp	→ reference no.
`GET /complaint/track/:ref?otp=`	track	public
`POST /complaint/account`	fileAccountComplaint	multipart, auth, no OTP
`POST /complaint/claim`	claim	{referenceNo, otp}
`GET /complaint/:id`	detail	auth, includes responses + ratings
`POST /response/respond`	respond	multipart follow-up
`POST /response/rate`	rateResponse	{responseId, rate}
`GET /discussion` , `GET /discussion/:id`	DiscussionService	public read
`POST /discussion/:id/comment`	comment	auth (guestName no longer sent)
`GET /poll?guestToken=` , `GET /poll/:id`	PollService	public read
`POST /poll/:id/vote`	vote	auth
`GET /notification` NEW	NotificationService.list	read + unread history, newest first
`POST /notification/read` NEW	NotificationService.markAllRead	HTTP fallback for the socket emit

9. Application Architecture

9.1 Layers

```
UI (screens / widgets)
     |  context.read/watch
State (AuthController : ChangeNotifier)          <-- provider
     |
Services (AuthService, ComplaintService, ...)    one per resource
     |
ApiClient (Dio + interceptors)  +  SocketService (Socket.IO)
     |
Backend  /api  +  Socket.IO channel
```

9.2 State & session

`AuthController` is the single app-wide store (registered as a `ChangeNotifierProvider` in `main.dart`). It mirrors the web `auth-store`:

- `token` (secure storage), `user` (prefs JSON), `isAuthenticated`, `isReady`.
- `notificationCount` and `notifications` — updated live from the socket.
- `bootstrap()` restores the persisted session on launch; `main.dart` does **not** await it, so the splash shows while it runs.
- The socket connects when a session is present and disconnects on logout / `force_logout`.
- **Remember me** (login checkbox, default on, persisted): when on, the token and profile are written to storage so `bootstrap()` can restore them, and the email is kept to pre-fill the field next time; when off, the session lives only in memory for that run and any previously-stored session is cleared. A freshly verified account always persists.

9.3 Routing & the auth redirect

`app_router.dart` uses `GoRouter` with `refreshListenable: authController`:

- `initialLocation: '/splash'`. The redirect holds on `/splash` until `auth.isReady`, then sends to `/home` (authed) or `/` (guest).
- Guest routes are top-level: `/`, `/login`, `/register`, `/verify-email`, `/file-complaint`, `/verify-complaint`, `/track`, `/discussions`, `/polls`, and the detail routes.
- The authed area is a `StatefulShellRoute.indexedStack` with five bottom-nav branches: **Home, Complaints, Discussions, Polls, Account**.
- Anything under `/home*` requires authentication; an authed user visiting an auth entry point is bounced to `/home`.
- Navigation was tuned so **every guest utility screen has a working back button** (the guest complaint flow uses `pushReplacement` to avoid returning into a stale form while still showing a back arrow).

10. Screens & Features

Screen	Route	Purpose
Splash	`/splash`	Shown while the session is restored; ePS wordmark on white + loader.
Landing	`/`	Guest home: hero, three service cards, verify / track / sign-up shortcuts.
Sign In	`/login`	Email + password, **Remember me** checkbox, **Forgot password?** link. 403 `notVerified` routes to Verify Email.
Forgot Password	`/forgot-password`	One screen, three steps: email → 6-digit code (verified on its own) → new password. Then back to Sign In (not auto-signed-in).
Sign Up	`/register`	Validated form (name, email, PH phone, password). Emails an OTP.
Verify Email	`/verify-email`	6-box OTP entry; on success the account is signed in.
File a Complaint	`/file-complaint`	Guest mode (OTP flow, optional anonymity) or account mode (no OTP), decided by auth state. Up to 5 evidence photos.
Verify Complaint	`/verify-complaint`	OTP entry → reference number, with a link to Track.
Track My Complaint	`/track`	Reference no. + access code → read-only complaint view.
Home	`/home`	Greeting, notification bell, service cards, "view my complaints".
My Complaints	`/home/complaints`	List with status chips, pull-to-refresh, "claim a complaint" dialog, file FAB.
Complaint Detail	`/my-complaints/:id`	Full thread, evidence, assigned offices, history, follow-up composer, per-response star rating + "Your satisfaction" per-office averages.
Discussions	`/home/discussions` / `/discussions`	Card list (open/closed, comment count).
Discussion Thread	`/discussions/:id`	Thread + comments; composer requires sign-in (else a `SignInPrompt`).
Polls	`/home/polls` / `/polls`	Card list (open/closed, voted, total votes).
Poll Vote	`/polls/:id`	Question + options; voting requires sign-in; results shown after voting.
Account	`/home/account`	Profile, server URL (read-only), sign out.

11. Real-time & Notifications

11.1 The socket

`SocketService` mirrors `src/boot/socket.js`:

- Connects only while authenticated; on connect it emits `authenticate` and `register` with the JWT.
- Listens for `loadNotifications(count, list)` — the server pushes this to the user's own socket(s) only.
- Listens for `force_logout` — surfaced as a callback that clears the session (JWT expired server-side).
- The Dart client delivers a multi-argument emit as a `List`; the handler reads `data[0]` (count) and `data[1]` (the notifications array).

11.2 Notification triggers (backend)

API CHANGE `ResponseController.js` now calls a `notifyUser` helper (creates a `Notification` row + `pushNotifications(io, receiverId)` from `utils/notifySocket.js`):

Event	Who is notified
An office **receives** a complaint (first time, or with a message)	the complainant
An office posts a **response**	the complainant
An office marks its part **complete**	the complainant ("resolved" / "...completed its part")
The citizen posts a **follow-up**	staff of the concerned office(s); falls back to super/admins if the complaint is not yet routed
The citizen **rates** a response	the staff member who wrote that response

Delivery is inherently logged-in-only: `pushNotifications` targets a live socket, and the app only opens the socket while authenticated. The `Notification` row persists, so an offline user sees the unread count on next login.

11.3 The notification tray

The socket only ever pushes the **unread** set. Opening the tray marks everything read, which makes the server re-push an empty list — which used to wipe the tray a frame after it opened. Fixed with a REST-backed history:

1. NEW `GET /api/notification` returns the last 30 notifications, **read and unread**, newest first. `POST /api/notification/read` marks all read.
2. `NotificationBell` (Home app bar): on tap it *snapshots* the socket's unread list for an instant render, then loads the full history from REST and replaces it.
3. It marks everything seen (socket emit + HTTP) **only after** the list is on screen, so read items stay visible.
4. The tray shows read vs. unread icons, timestamps, and supports pull-to-refresh.

11.4 Response ratings & office satisfaction

The `GET /complaint/:id` payload already includes `responseRatings` per response. The app derives, per office, the average of the citizen's stars on that office's responses in the complaint:

- **Complaint Detail** — interactive star editor per staff response, plus a "Your satisfaction" row of per-office average chips (`ComplaintBody._officeSatisfaction`).
- Rating a response re-fetches the detail so the average updates immediately.

11.5 Cross-client parity

Behaviour	Mobile	External web	Internal desk
Live unread badge (socket)	yes	yes (pre-existing)	yes
Notification triggers (§11.2)	yes	yes (same backend)	—
Tray with read+unread history, no wipe	yes	yes PATCHED	page snapshot
Per-office satisfaction average	yes	yes PATCHED	yes PATCHED
Complaint list / modal live refresh	no (badge only)	no (badge only)	yes — `complaints:changed`

API CHANGE `utils/notifySocket.js` also exports `broadcastComplaintsChanged(io, reason)` → a global `io.emit('complaints:changed')` , emitted from `ComplaintController` (validateOTP, createAccountComplaint, createStaffComplaint, reassignOffice, updateStatus) and `ResponseController` (receive, respond, complete, rate). Only the internal desk (`eps-www-quasar-internal/src/pages/ComplaintPage.vue`) listens for it and does an 800 ms-debounced refresh of its list + open modal. The citizen clients have no listener and are unaffected.

12. Design Decisions

12.1 Sign-in required to participate

Unlike the external web client, the mobile app requires an account to *participate* (not to view):

Action	Guest
Browse discussions / polls, track a complaint	allowed
Comment on a discussion	blocked — `SignInPrompt`
Vote on a poll	blocked — `SignInPrompt`
Post a complaint follow-up / rate a response	blocked (screen is account-only)

Rationale: on mobile the sign-in cost is tiny (autofill / biometric), and the app's value — push notifications, complaint history, one-tap follow-ups — only works with an account. Anonymous participation invites impersonation and ballot-stuffing that a public platform must guard against. `DiscussionService.comment` no longer sends a `guestName`.

12.2 Anonymous complaint filing kept

Filing a complaint *without* an account (the guest email-OTP flow, with an optional "report anonymously" toggle) is deliberately retained. This is a civic-participation and safety requirement — whistle-blowers and harassment reports — and government feedback channels are generally expected to accept anonymous submissions.

12.3 Splash screen

- **OS launch screen** via `flutter_native_splash` (config in `pubspec.yaml`): the official ePS wordmark on white. Source art copied from `eps-www-quasar-external/src/assets/epslogo.png` into `assets/branding/` as `eps_logo.png` (transparent) and `eps_logo_splash.png` (padded square). Regenerate with `dart run flutter_native_splash:create`.
- **In-app splash** (`splash_screen.dart`) shows while `AuthController.bootstrap()` runs, then the router redirects.

12.4 Launcher icon

The home-screen icon is the ePS "e" site mark — the same blue-circle-and-handshake artwork the internal and external web clients use as their favicon, re-rendered at high resolution from `epslogo.png`. It is generated by `flutter_launcher_icons` (config in `pubspec.yaml`) from `assets/branding/app_icon.png` and `app_icon_foreground.png`, with a white adaptive-icon background. Regenerate with `dart run flutter_launcher_icons`.

> The splash and icon output land in the git-ignored `android/` and `ios/` folders, so re-run both generators after any `flutter create .` (in addition to the §5.4 native tweaks).

13. Troubleshooting Log

Issues encountered while getting the first Android build to run, with the fix applied.

13.1 `flutter: command not found`

The SDK is not on `PATH`. See §5.1 — add `C:\src\flutter\bin` to the User PATH and reopen the terminal.

13.2 `Android sdkmanager not found` / `cmdline-tools component is missing`

Install *Android SDK Command-line Tools (latest)* from Android Studio's SDK Tools tab.

13.3 `Package ndk not found. Package 28.2.13676358 not found.` → `sdkmanager.bat … NTSTATUS 0xC0000409`

Cause: `path_provider_android` 2.3.x depends on `jni`, whose native code makes the Android Gradle Plugin try to download the NDK — via the machine's new `cmdline-tools` v23 shim, which hard-crashes.

Fix:

1. `pubspec.yaml` → `dependency_overrides: path_provider_android: 2.2.15` (removes `jni` from the tree entirely).
2. Comment out `ndkVersion = flutter.ndkVersion` in `android/app/build.gradle.kts` — nothing needs an NDK, and setting the property makes AGP provision one during configuration.
3. Install the compile-SDK platform once: `android sdk install "platforms;android-36"` (or via the SDK Platforms tab).
4. If the shim still bites, replace `%LOCALAPPDATA%\Android\Sdk\cmdline-tools\latest` with the classic "Command line tools only" package.

13.4 Kotlin build fails: `this and base files have different roots` / `Could not close incremental caches`

Cause: the Pub cache is on `C:` and the project on `D:`; Kotlin's incremental compiler cannot compute a relative path across drive roots.

Fix: add to `android/gradle.properties`:

```
kotlin.incremental=false
kotlin.incremental.useClasspathSnapshot=false
```

Then `flutter clean` (the failed run leaves corrupted caches) and rebuild. Alternative: move the Pub cache onto `D:` with `setx PUB_CACHE "D:\pub-cache"` and `flutter pub get`.

13.5 App runs but sign-in shows "Cannot reach the server"

Cause — one of two:

1. The APK was built without `--dart-define-from-file`, so it used the default `http://10.0.2.2:1000` — an emulator-only address a physical phone cannot reach. (The JSON file is *only* read when that flag is passed.)
2. The APK was built `--release` while pointed at a plain `http://` LAN address. Release builds have no cleartext exception, so Android blocks the connection and the app reports the same error.

Fix: rebuild with **both** the right define file **and** the right mode:

```
flutter build apk --debug --target-platform android-arm64 ^
  --dart-define-from-file=dart_define.device.json
```

Verify: the phone is on the same Wi-Fi; `EPS_API_BASE_URL` in `dart_define.device.json` equals the dev PC's current Wi-Fi IPv4; the backend is running on port 1000; Windows Firewall allows inbound TCP 1000 for `node.exe`.

13.6 `flutter doctor` shows "Android license status unknown"

A side effect of the new cmdline-tools shim. The standard SHA license files were written to `Sdk/licenses/`; doctor still flags it but it does not block builds.

13.7 Web build: "Wasm dry run failed … dart:html unsupported"

Only a warning. `flutter_secure_storage_web` uses `dart:html`, so the app cannot compile to WebAssembly; it falls back to JS compilation, which succeeds. Irrelevant for the mobile targets.

14. Change Log (this engagement)

Mobile app `eps-mobile-flutter`

- Full project scaffold: config, API client, socket service, models, services, state, router, theme, 17 screens, shared widgets.
- Splash: native launch screen + in-app splash; router held on `/splash` until session restored.
- Launcher icon: the ePS "e" site mark (matching the web clients' favicon) via `flutter_launcher_icons`.
- Sign-in gating for commenting, poll voting, follow-ups, ratings (`SignInPrompt`).
- Password recovery (3-step forgot-password screen) + "Remember me" on login (session persists only when checked).
- Complaints / discussions / polls lists rebuilt to the external web design — count line + search + one bordered card of separated rows, not one card per item.
- Notification system: `AppNotification` model, socket list parsing, `NotificationBell` tray with REST history and snapshot-then-load.
- Response rating display + "Your satisfaction" per-office averages in `ComplaintBody`.
- Back-button navigation fixes across the guest flow.
- Android build config: NDK dependency removed, cross-drive Kotlin fix, cleartext debug manifest, release signing wiring, `dart_define.*.json` per target.

Backend `eps-api-express-js`

- `utils/notifySocket.js`: added `broadcastComplaintsChanged(io, reason)`.
- `ResponseController.js`: `notifyUser` on receive / respond / complete / rate; `broadcastComplaintsChanged` on the same; staff notified when their response is rated.
- `ComplaintController.js`: `broadcastComplaintsChanged` on validateOTP, createAccountComplaint, createStaffComplaint, reassignOffice, updateStatus.
- New `NotificationController.js` + `routes/NotificationRoutes.js` (`GET /api/notification`, `POST /api/notification/read`), mounted in `server.js`.

External web client `eps-www-quasar-external`

- `auth-store.js`: `fetchNotifications()` / `markNotificationsRead()` using the new REST endpoints.
- `SiteHeader.vue`: bell dropdown now snapshots + loads history + marks read after (no more wipe), read/unread icons.
- `ComplaintPage.vue`: "your satisfaction" per-office average block.

Internal web client `eps-www-quasar-internal`

- `ComplaintPage.vue`: listens for `complaints:changed` (800 ms-debounced refresh of list + unassigned count + analytics + open modal); renders per-response citizen star ratings and per-office satisfaction averages in the Complaint Info modal.

15. Quick Reference Card

Task	Command
Static analysis	`flutter analyze`
Run on emulator	`flutter run --debug --dart-define-from-file=dart_define.json`
Run on phone (USB, LAN http)	`flutter run --debug -d <id> --dart-define-from-file=dart_define.device.json`
Run on web	`flutter run -d chrome --dart-define=EPS_API_BASE_URL=http://localhost:1000`
Sample APK for a phone (LAN http)	`flutter build apk **--debug** --target-platform android-arm64 --dart-define-from-file=dart_define.device.json`
Release APK (HTTPS prod only)	`flutter build apk --release --target-platform android-arm64 --dart-define-from-file=dart_define.prod.json`
Play Store bundle	`flutter build appbundle --release --dart-define-from-file=dart_define.prod.json`
Install last build	`flutter install --debug -d <id>` (match the mode you built; plain `flutter install` looks for the *release* APK)
List devices	`flutter devices`
Regenerate splash	`dart run flutter_native_splash:create`
Regenerate launcher icon	`dart run flutter_launcher_icons`
Clean (after a failed build)	`flutter clean && flutter pub get`
Re-scaffold native folders	`flutter create . --project-name eps_mobile --org ph.gov.sanfernandocity --platforms=android,ios,web` (then re-apply §5.4, re-run splash + icon)

> **`--debug` vs `--release`:** a LAN backend on plain `http://` requires a `--debug` (or `--profile`) build — only those manifests allow cleartext traffic. `--release` is for the HTTPS production API only.

> **Find the dev PC's LAN IP:** `ipconfig` → "Wireless LAN adapter Wi-Fi" → IPv4 Address. Put it in `dart_define.device.json` before building for a phone.
